

# Software Architecture & Implementation Blueprint

## Información del Documento

**Nombre del proyecto:** GravityLab - Simulador Interactivo de Sistemas Planetarios

**Autor / Equipo de desarrollo:** Eric Hernández Acosta

**Versión del documento:** 1.0

**Fecha:** 13 de marzo de 2026

**Institución / Organización:** Universidad Tecnológica de Pereira

---

## 1. Introducción

### 1.1 Propósito del Documento

Este documento define la arquitectura, estructura técnica y lineamientos de implementación del sistema **GravityLab**, un simulador interactivo de dinámica gravitacional que se ejecuta en navegador web.

El objetivo del documento es servir como guía para el análisis, diseño e implementación del sistema, estableciendo los principios de arquitectura, el stack tecnológico, las responsabilidades de cada componente y las restricciones del proyecto

Este documento también puede ser utilizado como base para el desarrollo asistido por herramientas de inteligencia artificial, garantizando que las decisiones de arquitectura y diseño se mantengan alineadas con los objetivos del sistema.

### 1.2 Alcance del Sistema

#### Descripción del problema

La dinámica gravitacional entre múltiples cuerpos es uno de los fenómenos fundamentales estudiados en cursos de **mecánica clásica**, astrofísica y dinámica orbital. Sin embargo, su comprensión suele basarse principalmente en modelos matemáticos abstractos, ecuaciones diferenciales y aproximaciones analíticas.

Aunque estos modelos permiten describir formalmente el movimiento de los cuerpos bajo interacción gravitacional, resulta difícil para los estudiantes construir una **intuición física clara** sobre cómo evolucionan sistemas complejos de múltiples cuerpos.

Además, las escalas espaciales y temporales de los sistemas planetarios reales son tan grandes que los fenómenos dinámicos ocurren en escalas que exceden la percepción directa humana. Como resultado, muchos procesos orbitales —como resonancias, perturbaciones gravitacionales, colisiones o configuraciones caóticas— son difíciles de visualizar y comprender de forma intuitiva.

Por estas razones, las herramientas de simulación interactiva representan un recurso pedagógico valioso para apoyar la enseñanza de estos conceptos, permitiendo experimentar con configuraciones físicas de manera visual e interactiva.

**Objetivo principal del sistema:**

El objetivo del sistema es desarrollar un **simulador interactivo de sistemas gravitacionales de múltiples cuerpos** que permita a los usuarios crear, modificar y observar la evolución dinámica de sistemas planetarios bajo la acción de la gravedad.

El simulador permitirá:

- crear cuerpos celestes con diferentes masas
- definir condiciones iniciales de posición y velocidad
- observar trayectorias orbitales en tiempo real
- analizar interacciones gravitacionales entre múltiples cuerpos
- registrar eventos dinámicos como colisiones
- modificar parámetros físicos durante la simulación

El sistema está orientado principalmente a **estudiantes universitarios de física o áreas afines**, aunque también busca ser una herramienta útil para **divulgación científica**.

## Alcance del proyecto

El proyecto incluye:

- Desarrollo de un motor físico de simulación gravitacional N-cuerpos
- Implementación de un canvas interactivo en 3D para visualización del sistema
- Creación de un sistema de interacción que permita al usuario agregar, modificar o eliminar cuerpos
- Implementación de controles de simulación (iniciar, pausar, reiniciar)
- Visualización de trayectorias orbitales (tracks)
- Panel informativo con parámetros físicos en tiempo real
- Registro de eventos dinámicos como colisiones entre cuerpos
- Control del factor de velocidad temporal de la simulación

Fuera del alcance del proyecto:

- simulaciones relativistas
- simulaciones con miles de partículas
- integración con bases de datos externas
- simulaciones astrofísicas de alta precisión profesional
- renderizado físico avanzado de superficies planetarias

## 2. Contexto de la Misión (Mission Context)

### 2.1 Descripción General

GravityLab es una aplicación web interactiva que permite simular sistemas gravitacionales compuestos por múltiples cuerpos en un entorno tridimensional.

El usuario podrá crear cuerpos celestes directamente en el espacio de simulación, asignarles propiedades físicas como masa, posición y velocidad inicial, y observar cómo evolucionan sus trayectorias bajo la influencia de la interacción gravitacional mutua.

El motor físico del simulador implementa la ley de gravitación universal:

$$F = G \frac{m_1 m_2}{r^2}$$

La simulación calcula en cada instante las fuerzas gravitacionales entre todos los cuerpos del sistema y actualiza sus velocidades y posiciones de forma iterativa.

El sistema soportará hasta ocho cuerpos simultáneamente, permitiendo explorar configuraciones como:

- sistemas planetarios
- sistemas binarios
- configuraciones caóticas de tres cuerpos
- colisiones gravitacionales
- formación y perturbación de órbitas

## 2.2 Contexto educativo

El simulador está diseñado para apoyar la enseñanza de conceptos fundamentales como:

- dinámica orbital
- leyes de Kepler
- interacción gravitacional
- estabilidad de sistemas planetarios
- conservación de energía
- comportamiento caótico en sistemas N-cuerpos

Además, el sistema permitirá manipular el **factor de velocidad temporal**, lo que facilita observar procesos dinámicos que en la realidad ocurren en escalas temporales extremadamente largas.

## 3. Actores del Sistema

### Descripción:

Usuario final que interactúa directamente con el simulador a través de la interfaz gráfica en el navegador.

### Responsabilidades:

- Crear cuerpos celestes mediante un panel de configuración
- Definir parámetros físicos iniciales (masa, posición, velocidad, etc.)
- Controlar la simulación (iniciar, pausar, reiniciar)
- Activar o desactivar visualizaciones (trayectorias, vectores, etc.)
- Analizar el comportamiento dinámico del sistema

### Nivel de acceso:

Total sobre la simulación

## Motor de Simulación Física

### Descripción:

Módulo interno encargado de calcular las interacciones gravitacionales entre los cuerpos.

**Responsabilidades:**

- Calcular fuerzas gravitacionales entre todos los cuerpos
- Actualizar velocidades y posiciones en cada iteración
- Detectar colisiones
- Aplicar reglas de fusión entre cuerpos
- Calcular magnitudes físicas en tiempo real

**Motor de Renderizado (Visualización)****Descripción:**

Encargado de representar gráficamente el sistema en el canvas 3D.

**Responsabilidades:**

- Dibujar cuerpos celestes
- Representar trayectorias (tracks)
- Actualizar posiciones en tiempo real
- Mostrar vectores (velocidad, fuerza si se activa)
- Mantener rendimiento visual

**Sistema de Interfaz de Usuario (UI)****Descripción:**

Gestiona la interacción entre el usuario y el sistema.

**Responsabilidades:**

- Mostrar panel de creación de cuerpos
- Mostrar controles de simulación
- Mostrar indicadores físicos en tiempo real
- Gestionar eventos del usuario (clicks, inputs, botones)

## 4. Requisitos del Sistema

### 4.1 Requisitos Funcionales

#### Creación y gestión de cuerpos

RF1. El sistema debe permitir crear cuerpos celestes mediante un panel de configuración.

RF2. El sistema debe permitir definir los siguientes parámetros al crear un cuerpo:

- masa
- posición inicial (x, y, z)
- velocidad inicial (vector)
- radio (opcional, para visualización y colisión)

RF3. El sistema debe permitir eliminar cuerpos existentes.

RF4. El sistema debe limitar el número máximo de cuerpos a 8 simultáneamente.

#### Simulación física

RF5. El sistema debe calcular la interacción gravitacional entre todos los cuerpos utilizando la ley:

$$F = G \frac{m_1 m_2}{r^2}$$

RF6. El sistema debe actualizar las posiciones y velocidades de los cuerpos en tiempo real.

RF7. El sistema debe permitir modificar la velocidad de la simulación mediante un controlador de tiempo.

RF8. El sistema debe permitir iniciar, pausar y reiniciar la simulación.

#### Colisiones

RF9. El sistema debe detectar colisiones entre cuerpos.

RF10. En caso de colisión, el sistema debe aplicar un modelo de **fusión**, donde:

- las masas se combinan

- se conserva el momento lineal
- se genera un nuevo cuerpo resultante

### **Visualización**

RF11. El sistema debe representar los cuerpos en un entorno 3D interactivo.

RF12. El sistema debe permitir activar o desactivar la visualización de trayectorias orbitales.

RF13. El sistema debe mostrar el movimiento de los cuerpos en tiempo real.

RF14. El sistema debe permitir visualizar vectores físicos (opcionalmente), como velocidad.

### **Información en tiempo real**

RF15. El sistema debe mostrar información individual de cada cuerpo:

- velocidad
- masa
- posición

RF16. El sistema debe mostrar indicadores físicos globales del sistema:

- energía cinética
- energía potencial
- energía total
- momento angular

### **Registro de eventos**

RF17. El sistema debe registrar eventos de colisión entre cuerpos.

RF18. El sistema debe mostrar información básica de cada evento registrado (cuerpos involucrados, tiempo de simulación).

## **4.2 Requisitos No Funcionales**

RNF1. El sistema debe ejecutarse en un navegador web moderno.

RNF2. El sistema debe mantener un rendimiento fluido con hasta 8 cuerpos activos.

RNF3. El sistema debe responder a interacciones del usuario en tiempo real.

RNF4. El sistema debe ser visualmente claro y comprensible para fines educativos.

RNF5. El sistema debe ser modular para facilitar futuras ampliaciones.

RNF6. El sistema debe ser compatible con despliegue en plataformas web como GitHub Pages.

## **5. Stack Tecnológico y Restricciones (Tech Stack s Constraints)**

### **5.1 Descripción**

El sistema GravityLab será desarrollado como una aplicación web interactiva que se ejecuta directamente en el navegador, utilizando tecnologías estándar de desarrollo frontend.

El enfoque tecnológico prioriza:

- rendimiento en tiempo real
- visualización 3D fluida
- simplicidad de implementación
- mantenibilidad del código

### **5.2 Tecnologías Principales Lenguaje / Framework**

HTML5

CSS3

JavaScript (ES6+)

#### **Motor gráfico 3D**

- Three.js

#### **Justificación:**

facilita la

implementación de

escenas 3D en

navegador



- abstrae la  
complejidad de  
WebGL
- permite  
trabajar con  
cámaras, luces  
y objetos  
fácilmente
- ideal para  
simulaciones  
educativas  
interactivas

## **Renderizado**

- Canvas WebGL  
(a través de  
Three.js)

## **Entorno de ejecución**

Navegador web moderno (Chrome, Firefox, Edge)

Desarrollo local en Antigravity

## **6. Arquitectura del Sistema**

### **6.1 Arquitectura General**

El sistema seguirá una arquitectura modular por capas, adaptada a aplicaciones interactivas en tiempo real.

Se definen cuatro componentes principales:

#### **Capa de Presentación**

Responsable de la interacción con el usuario.

Incluye:

- panel de creación de planetas
- controles (play, pausa, reset)

- sliders (velocidad de simulación)
- visualización de datos físicos

### **Capa de Simulación (Core Physics Engine)**

Es el corazón del sistema.

Responsable de:

- calcular fuerzas gravitacionales
- actualizar posiciones y velocidades
- detectar colisiones
- aplicar fusión de cuerpos

Se basa en la ley:

$$F = G \frac{m_1 m_2}{r^2}$$

### **Motor de Renderizado (Graphics Engine)**

Encargado de la representación visual en 3D.

Funciones:

- renderizar cuerpos en escena
- actualizar posiciones en cada frame
- dibujar trayectorias
- manejar cámara y perspectiva

Implementado con Three.js

### **Gestor de Simulación (Simulation Controller)**

Actúa como intermediario entre UI, física y render.

Responsabilidades:

- controlar el estado (play/pause/reset)
- gestionar el tiempo de simulación
- coordinar actualización física + render
- manejar eventos (colisiones, creación de cuerpos)

Se recomienda el uso de los siguientes patrones arquitectónicos:

- **Arquitectura Limpia (Clean Architecture):** organiza el sistema en capas concéntricas donde el dominio está en el centro y las dependencias apuntan hacia él. Permite independencia de frameworks, bases de datos e interfaces.
- **MVC (Model View Controller):** divide el sistema en tres componentes. El *Model* gestiona los datos y reglas del dominio, la *View* representa la información al usuario y el *Controller* maneja las acciones del usuario y coordina las operaciones.
- **Arquitectura por Capas:** separa el sistema en niveles jerárquicos (presentación, aplicación, dominio e infraestructura) con responsabilidades bien definidas.

## 7. Modelo de Dominio y Datos

### 7.1 Descripción

El modelo de dominio define las entidades principales del simulador y sus relaciones, representando el comportamiento físico de un sistema gravitacional de múltiples cuerpos.

El sistema se basa en una estructura donde cada cuerpo celeste interactúa con todos los demás mediante la ley de gravitación universal:

$$F = G \frac{m_1 m_2}{r^2}$$

### 7.2 Entidades principales

Definir:

#### Entidad: Body (Cuerpo Celeste)

Representa un objeto físico dentro del sistema.

**Atributos:**

id → identificador único

mass → masa

position → vector 3D (x, y, z)

velocity → vector 3D  
acceleration → vector 3D

radius → radio (para renderizado y colisiones)

color → color del cuerpo

trail → historial de posiciones (trayectoria)

**Responsabilidades:**

almacenar estado físico

actualizar su movimiento

registrar su trayectoria

#### Entidad: Simulation

Representa el sistema completo.

**Atributos:**

- bodies → lista de cuerpos
- G → constante gravitacional (escalada)
- timeScale → factor de velocidad
- isRunning → estado (activo/pausado)
- collisionLog → registro de colisiones

**Responsabilidades:**

- gestionar todos los cuerpos
- coordinar el cálculo físico
- manejar eventos globales

**Entidad: PhysicsEngine**

Encargado del cálculo físico.

**Responsabilidades:**

- calcular fuerzas entre cuerpos
- aplicar integración numérica (Verlet)
- detectar colisiones
- aplicar fusión

**Entidad: Renderer (Three.js)**

Encargado de la visualización.

**Responsabilidades:**

- renderizar esferas (cuerpos)
- actualizar posiciones visuales
- dibujar trayectorias
- gestionar cámara libre

**Entidad: UIController**

Interfaz con el usuario.

**Responsabilidades:**

- crear cuerpos desde panel
- actualizar parámetros
- mostrar información en tiempo real
- manejar botones

### 7.3 Relaciones

Una Simulation contiene múltiples Body

El PhysicsEngine actúa sobre los cuerpos

El Renderer visualiza los cuerpos

El UIController interactúa con la Simulation

## 8. Interfaz de Usuario y Experiencia (UI/UX)

### 8.1 Descripción general

La interfaz de usuario de GravityLab está diseñada para facilitar la interacción intuitiva con el sistema de simulación, permitiendo al usuario crear, modificar y analizar sistemas gravitacionales en un entorno tridimensional.

El diseño prioriza:

claridad visual

acceso rápido a controles

visualización simultánea de datos físicos

mínima obstrucción del espacio de simulación

### 8.2 Estructura de la interfaz

La interfaz se divide en tres componentes principales:

#### 1. Área de simulación (Canvas 3D)

- Ocupa la mayor parte de la pantalla
- Renderiza el sistema con Three.js
- Permite interacción directa con la cámara

#### Controles de cámara:

- rotación → mouse (click + arrastrar)
- zoom → scroll
- desplazamiento → arrastre (según configuración de cámara orbital)

#### 2. Panel lateral de control

Ubicado en el lado derecho o izquierdo de la pantalla.

Contiene:

#### Controles de simulación

- Iniciar
- Pausar

- Reiniciar

### **Control de tiempo**

- Slider de velocidad de simulación (timeScale)

### **Gestión de cuerpos**

- Botón: "Crear planeta"
- Lista de cuerpos existentes (opcional futuro)
- Botón eliminar cuerpo

### **Visualización**

- Toggle: mostrar/ocultar trayectorias
- Toggle: mostrar vectores (opcional)

## **3. Panel lateral de datos físicos**

Ubicado junto al panel de control o integrado en él.

Muestra en tiempo real:

### **Datos globales del sistema**

- Energía cinética total
- Energía potencial total
- Energía total
- Momento angular

### **Datos por cuerpo (al seleccionar uno)**

- Masa
- Velocidad
- Posición
- Distancia a otros cuerpos (opcional)

## **8.3 Modal de creación de cuerpos**

El sistema utiliza una **ventana emergente (modal)** para la creación de nuevos cuerpos.

### **Campos del formulario:**

- Masa
- Posición inicial (x, y, z)
- Velocidad inicial (vx, vy, vz)
- Radio (opcional)
- Color

### **Comportamiento:**

- El usuario abre el modal con el botón "Crear planeta"
- Ingresar los parámetros
- Al confirmar, el cuerpo se agrega al sistema
- Se cierra el modal automáticamente

### **8.4 Interacción visual**

#### **Trayectorias (tracks)**

- Activables/desactivables
- Representan el historial de movimiento

#### **Colisiones**

- Se registran en el sistema
- Opcionalmente pueden mostrarse como eventos visuales

### **8.5 Flujo de interacción del usuario**

1. Usuario abre el simulador
2. Presiona "Crear planeta"
3. Ingresar parámetros en el modal
4. Se agrega el cuerpo al sistema
5. Inicia la simulación
6. Observa el comportamiento dinámico
7. Ajusta velocidad o agrega más cuerpos
8. Analiza datos físicos en tiempo real

## **G. Estándares de Desarrollo**

### **G.1 Reglas de Estilo**

- PEP8 para Python
- camelCase para JavaScript
- snake\_case para Python

### **G.2 Patrones de Diseño**

#### **Patrones recomendados:**

- **Singleton:** garantiza que una clase tenga una única instancia durante la ejecución del programa. Es útil para manejar recursos compartidos como configuraciones globales, conexiones a bases de datos o gestores de logs.

- **Factory:** permite crear objetos sin especificar exactamente la clase del objeto que se va a instanciar. Centraliza la lógica de creación de objetos y facilita modificar el tipo de objeto creado sin cambiar el resto del sistema.
- **Observer:** define una relación uno-a-muchos entre objetos. Cuando un objeto cambia de estado, todos los objetos suscritos reciben automáticamente una notificación. Es ampliamente utilizado en sistemas de eventos, interfaces gráficas y arquitecturas reactivas.
- **Strategy:** permite definir una familia de algoritmos intercambiables. Cada algoritmo se encapsula en una clase diferente y el sistema puede cambiar dinámicamente entre ellos según el contexto.

Estos patrones ayudan a resolver problemas recurrentes de diseño de software y promueven una arquitectura flexible, modular y fácil de mantener.

## 10. Flujo de Trabajo y Checkpoints (Agent Workflow)

Durante el desarrollo asistido por inteligencia artificial se deben establecer puntos de validación que permitan verificar que las decisiones de diseño, arquitectura y lógica del sistema se mantengan alineadas con los objetivos del proyecto. Estos puntos de control ayudan a prevenir errores tempranos, mejorar la calidad del software y asegurar que cada etapa del desarrollo sea revisada antes de avanzar a la siguiente fase.

### Checkpoint 1

Validación del modelo de datos, verificando que las entidades, atributos y relaciones representen correctamente el dominio del problema y que el diseño de la base de datos sea consistente y normalizado.

### Checkpoint 2

Validación de la lógica del núcleo del sistema, revisando los algoritmos principales, las estructuras de datos utilizadas y la coherencia entre los componentes del dominio.



### Checkpoint 3

Validación de la arquitectura del software, confirmando que las capas del sistema, los módulos y los patrones arquitectónicos definidos se implementan correctamente.

### Checkpoint 4

Validación de la interfaz de usuario, evaluando la usabilidad, organización de los componentes visuales y coherencia con los requisitos funcionales.

### Checkpoint 5

Validación de las pruebas automatizadas, comprobando que existen pruebas unitarias e integradas suficientes para garantizar la estabilidad y confiabilidad del sistema.